

University of Messina

Bachelor of Data Analysis



Identify best features to build the best model able to predict the different emotional states.

Data Mining– Project Report

Academic Year 2023/2024

Student ():

J.A.D. Serena (531401)

Table of Contents

Table of Contents	Error! Bookmark not defined.
Introduction	Error! Bookmark not defined.
Scope of the study	2
___Significance of the study.....	3
___Materials and Methods	3
Results and Discussion	5
Conclusion.....	Error
! Bookmark not defined.	
References.....	6

introduction

With advancements in technology and data analytics, the development of an effective health monitoring system has become a key focus in the field of healthcare. This project aims to contribute to the enhancement of health monitoring through the utilization of classification algorithms and ensemble learning techniques.

The dataset used in this study contains various health-related features, and the task is to predict health conditions based on these features. The need for accurate predictive models arises from the increasing complexity of health data and the desire to move towards personalized medicine. As individuals generate vast amounts of health-related data through wearables, sensors, and other monitoring devices, the challenge lies in extracting meaningful insights from this data to inform healthcare decisions. I tried the integration of classification algorithms and ensemble learning techniques enhance the accuracy and efficiency of a health monitoring system aim to investigate the efficacy of combining multiple models to create an ensemble that outperforms individual models in predicting health outcomes.

Scope of study

The dataset in attachment contains some features extracted by processing physiological responses of 186 healthy subjects. The aim of the study was measuring cardiovascular, respiratory, and electrodermal responses to positive stimuli that differed in approach intensity. Specifically, this study examined how high approach and low approach positive emotions buffer stress responses. Also, it was examined whether reactivity to anger (a high approach negative emotion) vs. threat would interact with high approach positive affect. in this report, we will explore the methodology employed in developing the health monitoring system, including data preprocessing, feature engineering, and the application of various classification algorithms. Results from individual models, ensemble learning techniques, and feature importance analyses will be presented and discussed. The report aims to provide a comprehensive understanding of the implemented system,

Significance of the Study

Additionally, I seek to identify the key features and patterns crucial for accurate health predictions.

Material and Method

Data preprocessing:

I imported all the necessary datasets then I started by loading the dataset using the Pandas library.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report, confusion_matrix
from sklearn.feature_selection import SelectFromModel
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import RandomOverSampler
from sklearn.ensemble import VotingClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import LabelEncoder
from sklearn.inspection import permutation_importance
```

Explored the data by printing the num columns and rows, first 5 lines and data types. I found that all data are numerical. Then I Checked for any duplicate entries and null values in the dataset. I found 2 duplicates, and I removed them to ensure the integrity of the data.

```
# Load the dataset
df = pd.read_csv("/content/drive/MyDrive/Dataset_Study2.csv")

df.shape

df.head()

df.dtypes

df.columns

df.duplicated().sum()

df.isnull().sum()

# Remove duplicates
df = df.drop_duplicates()

df.shape

df['Label'].value_counts()
```

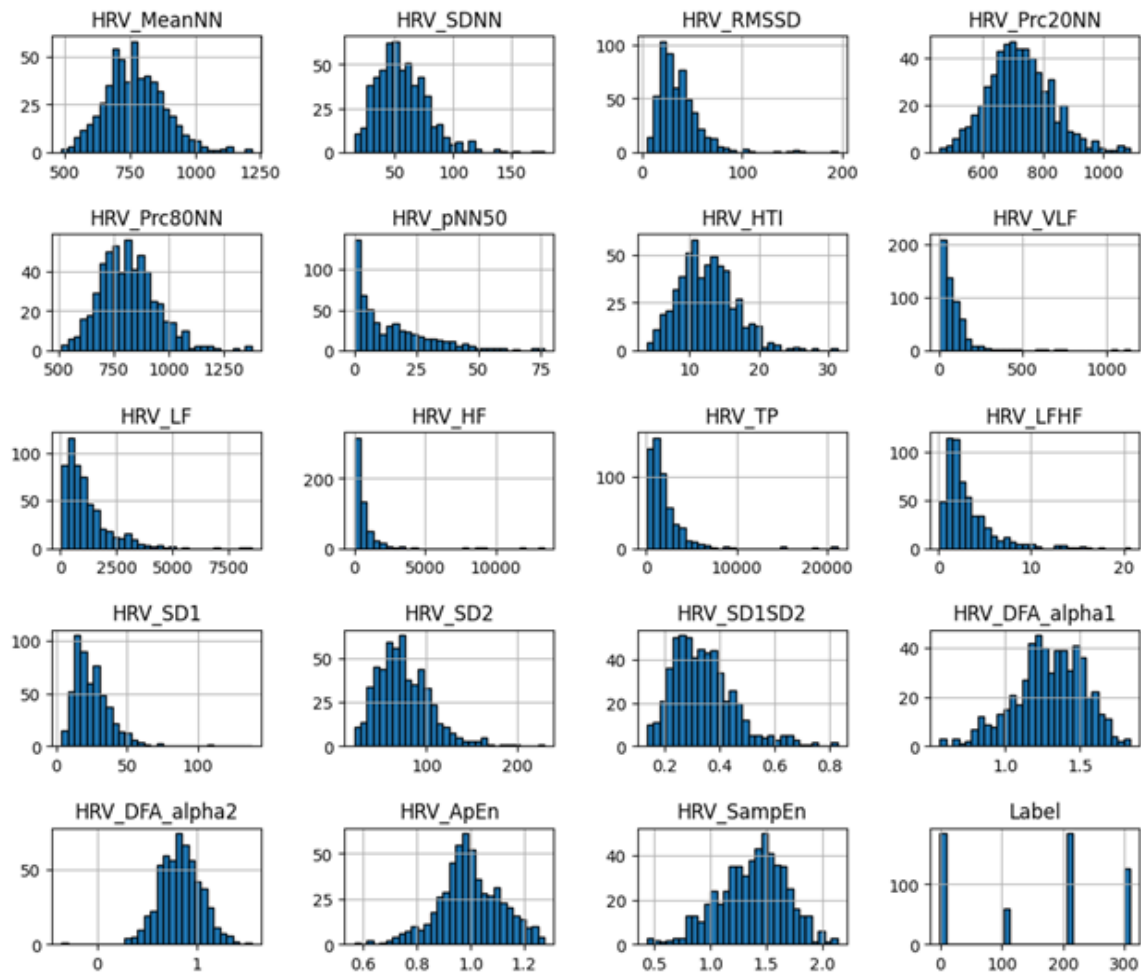
Also, there is a column 'label', so we can perform Supervised Learning Models as they Trained on labeled data with input-output pairs.

Exploratory Data Analysis (EDA):

To understand the distribution of variables I used histograms for univariate analysis and box plots to identify outliers.

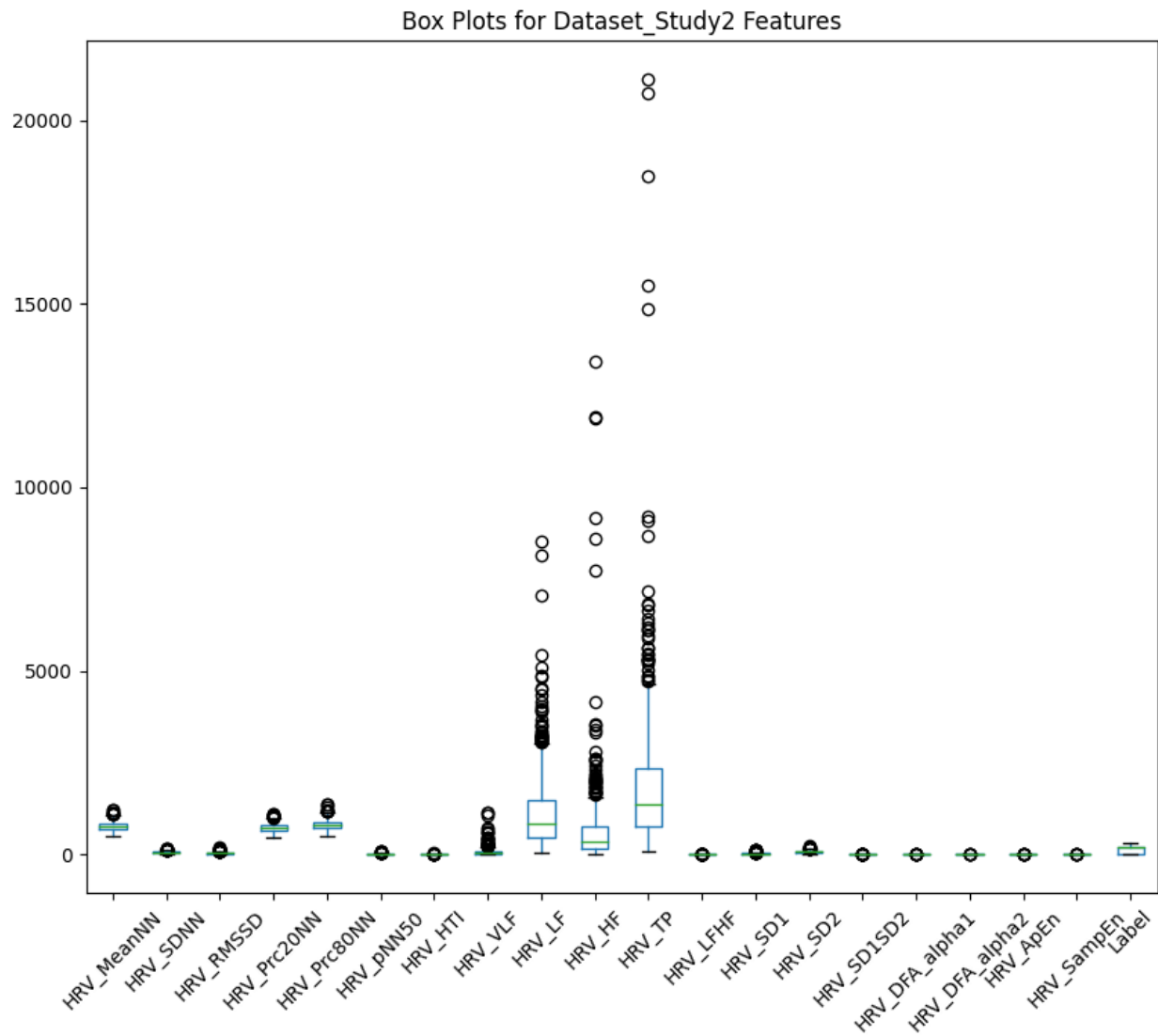
```
#data visualisation
df.hist(figsize=(12, 10), bins=30, edgecolor="black")
plt.subplots_adjust(hspace=0.7, wspace=0.4)

#box plot
plt.figure(figsize=(10, 8))
df.boxplot()
plt.title('Box Plots for Dataset_Study2 Features')
plt.xticks(rotation=45)
plt.grid(False)
plt.show()
```



-Majority of Features have a good variance.

below graph it shows the outliers present in the dataset,



Feature Engineering and Correlation Analysis:

-Feature Selection and Removal:

While exploring the above distribution I noticed that as HRV_LFHF and HRV_SD1SD2 exist we no longer need HRV_LF, HRV_HF, HRV_SD1 and HRV_SD2 so I removed specific features.

```
#drop unnecessary column as they exist
df_clean = df.copy()
df_clean = df_clean.drop('HRV_LF', axis=1)
df_clean = df_clean.drop('HRV_HF', axis=1)
df_clean = df_clean.drop('HRV_SD1', axis=1)
df_clean = df_clean.drop('HRV_SD2', axis=1)

df_clean.shape

#after cleaning data do corr matrix
```

-Also, the dataset contains some clinical data if I handle outliers there is a possibility that important information will be lost. So, the model will fail to do its task. So, I decided not to handle outliers.

Correlation Matrix Computation and Visualization:

I Computed the correlation matrix to understand the relationships between features. Visualize the matrix to identify highly correlated or irrelevant features

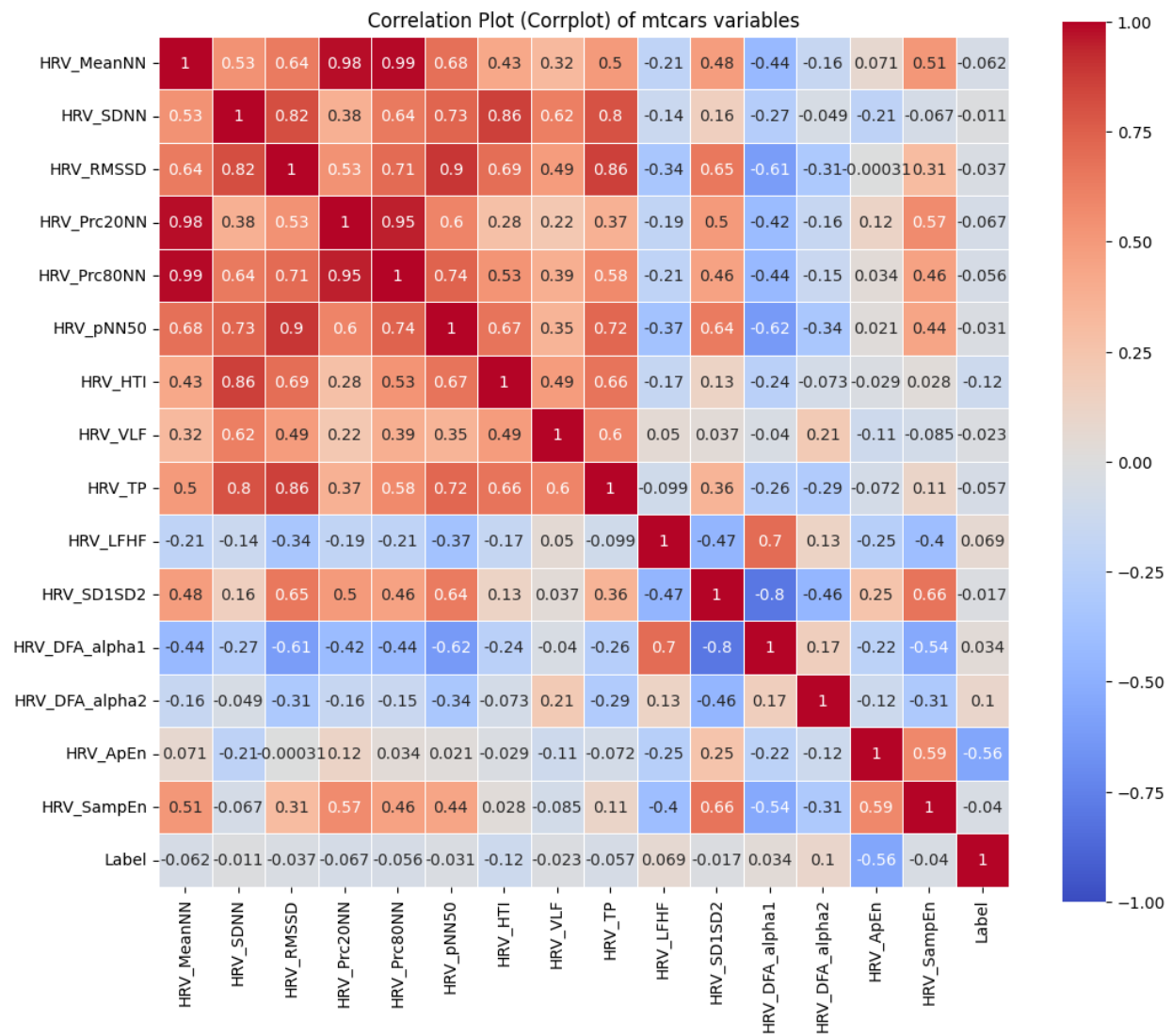
```
# Computing correlation matrix
corr_matrix = df_clean.corr(numeric_only=True)

# Plot heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, cmap='coolwarm', annot=True, vmin=-1, vmax=1, linewidths=0.5, square=True)
plt.title('Correlation Plot (Corrplot) of mtcars variables')
plt.show()
```

```
[ ] corr_matrix.iloc[-1,:]
```

HRV_MeanNN	-0.062014
HRV_SDNN	-0.011449
HRV_RMSSD	-0.037039
HRV_Prc20NN	-0.066870
HRV_Prc80NN	-0.056468
HRV_pNN50	-0.030728
HRV_HTI	-0.121975
HRV_VLF	-0.022872
HRV_TP	-0.057028
HRV_LFHF	0.068823
HRV_SD1SD2	-0.017484
HRV_DFA_alpha1	0.033677
HRV_DFA_alpha2	0.099838
HRV_ApEn	-0.557224
HRV_SampEn	-0.040473
Label	1.000000

Name: Label, dtype: float64



Class Imbalance Handling:

Then I did class imbalance Handling in the dataset as it is crucial because it can significantly impact the performance and reliability of machine learning models, particularly in classification tasks. Class imbalance is the situation where the number of instances in one class is significantly higher or lower than the number of instances in another class. This imbalance can negatively impact the performance of machine learning models, as they might become biased towards predicting the majority class. There are several ways to handle this.

SMOTE :(Synthetic Minority Over-sampling Technique) is a technique used to Creating synthetic instances of the minority class in the context of binary classification problems. repeating the

synthetic instance generation process for each minority class instance, SMOTE increases the number of instances in the minority class, making it more balanced with the majority class.

□ Random Undersampling: Randomly removing instances from the majority class to balance the class distribution. This is a simple technique, but it may lead to loss of information.

Random Oversampling: Randomly duplicating instances from the minority class to balance the class distribution.

-I decided to use each of these 3 methods.

```
# Class Imbalance Handling (Using SMOTE)
#X = df_clean.drop('Label', axis=1)
#y = df_clean['Label']
#smote = SMOTE(random_state=0)
#X_resampled, y_resampled = smote.fit_resample(X, y)

# Class Imbalance Handling (Using Random Under-Sampling)
X = df_clean.drop('Label', axis=1)
y = df_clean['Label']
#rus = RandomUnderSampler(random_state=0)
#X_resampled, y_resampled = rus.fit_resample(X, y)

# Random Over-Sampling
ros = RandomOverSampler(random_state=0)
X_resampled, y_resampled = ros.fit_resample(X, y)
```

Train-Test Split and Standardization: the process of splitting the dataset into training and testing sets is performed to assess how well the model generalizes to new data. By dividing the dataset into training and testing sets, we can train the model on one portion of the data and evaluate its performance on another, independent portion. The function 'train_test_split' is from a machine learning library (e.g., scikit-learn). This function randomly assigns samples to the training and testing sets. Separate the features (X) from the labels (y) in both the training and testing sets. Also, train-test split helps prevent overfitting by providing an independent dataset for model evaluation.

standardization to ensure consistent scales for features. standardization is the process of rescaling features to have a mean of 0 and a standard deviation of 1. It ensures that all features have the same scale, which is important for certain machine learning algorithms.

First, I Identified numerical features in dataset that need to be standardized. Then I Applied the standardization transformation separately to the training and testing sets. mean and standard

deviation using only the training set to prevent information leakage from the testing set. By following these processes, I created a well-prepared dataset for training and testing machine learning models.

```
# Splitting the data into training and testing sets (80-20 split)
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled, test_size=0.2, random_state=0)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Trying ML models:

Model- refers to a mathematical representation or algorithm that captures patterns, relationships, or trends in data. A machine learning model is trained on historical(input)data to learn patterns and make predictions or decisions without being explicitly programmed.

I've chosen a diverse set of models and ensemble techniques to address different aspects of monitoring the problem. I used ensemble methods like Random Forest, Bagging, Stacking, and Boosting which are particularly useful for improving overall predictive performance and handling complex relationships within the data.

Random Forest: an ensemble learning method that builds multiple decision trees and merges them together to get a more accurate and stable prediction. robust against overfitting and tends to perform well on various types of data.

```
# Logistic Regression with hyperparameter tuning
lr_param_grid = {'C': [0.001, 0.01, 0.1, 1, 10, 100], 'penalty': ['l2']}
lr_grid_search = GridSearchCV(LogisticRegression(random_state=0, max_iter=1000), lr_param_grid, cv=5, scoring='accuracy')
lr_grid_search.fit(X_train_scaled, y_train)
lr_best_model = lr_grid_search.best_estimator_

# Print results for Logistic Regression
print(f"Best Hyperparameters for Logistic Regression: {lr_grid_search.best_params_}")

print_model_evaluation(lr_best_model, X_train_scaled, y_train, X_test_scaled, y_test, 'Logistic Regression')
```

Random Forest is suitable for its ability to handle a large number of features and complex relationships within the data. It's also effective in dealing with imbalanced datasets and provides feature importance for better interpretability.

Logistic Regression: a linear model used for binary classification. It's a simple and interpretable algorithm that models the probability of the occurrence of an event. Logistic Regression is commonly used in health-related studies when the outcome is binary (disease presence or absence). provides coefficients that can be interpreted to understand the impact of features on the outcome.

```
# Random Forest with hyperparameter tuning
rf_param_grid = {'n_estimators': [50, 100, 200], 'max_depth': [None, 10, 20], 'min_samples_split': [2, 5, 10]}
rf_grid_search = GridSearchCV(RandomForestClassifier(random_state=0), rf_param_grid, cv=5, scoring='accuracy')
rf_grid_search.fit(X_train_scaled, y_train)
rf_best_model = rf_grid_search.best_estimator_

# Print results for Random Forest
print(f"Best Hyperparameters for Random Forest: {rf_grid_search.best_params_}")

print_model_evaluation(rf_best_model, X_train_scaled, y_train, X_test_scaled, y_test, 'Random Forest')
```

K-Nearest Neighbors (KNN): is a non-parametric, instance-based learning algorithm that classifies new instances based on their similarity to existing instances in the training data. KNN is well-suited for health monitoring it doesn't assume a specific functional form for the underlying distribution. It can capture complex patterns in the data and is particularly useful when instances with similar features tend to have similar outcomes.

```
# K-Nearest Neighbors with hyperparameter tuning
knn_param_grid = {'n_neighbors': [5, 10, 15], 'weights': ['uniform', 'distance']}
knn_grid_search = GridSearchCV(KNeighborsClassifier(), knn_param_grid, cv=5, scoring='accuracy')
knn_grid_search.fit(X_train_scaled, y_train)
knn_best_model = knn_grid_search.best_estimator_

# Print results for K-Nearest Neighbors
print(f"Best Hyperparameters for K-Nearest Neighbors: {knn_grid_search.best_params_}")

print_model_evaluation(knn_best_model, X_train_scaled, y_train, X_test_scaled, y_test, 'K-Nearest Neighbors')
```

Support Vector Machine (SVM): a supervised machine learning algorithm that can be used for both classification and regression tasks. SVM finds the optimal hyperplane that separates data into classes in a high-dimensional space. SVM is effective in high-dimensional spaces, making it suitable for health monitoring datasets with many

features. It's also capable of handling non-linear relationships using kernel functions.

```
# Support Vector Machine with hyperparameter tuning
svm_param_grid = {'C': [0.1, 1, 10], 'kernel': ['linear', 'rbf'], 'gamma': ['scale', 'auto']}
svm_grid_search = GridSearchCV(SVC(), svm_param_grid, cv=5, scoring='accuracy')
svm_grid_search.fit(X_train_scaled, y_train)
svm_best_model = svm_grid_search.best_estimator_

# Print results for Support Vector Machine
print(f"Best Hyperparameters for Support Vector Machine: {svm_grid_search.best_params_}")
print_model_evaluation(svm_best_model, X_train_scaled, y_train, X_test_scaled, y_test, 'Support Vector Machine')
```

Decision Tree: Decision Trees recursively split the data based on feature conditions, creating a tree-like structure to make decisions. They are intuitive and easy to interpret. They can capture non-linear relationships and are useful for identifying important features.

```
# Decision Tree with hyperparameter tuning
dt_param_grid = {'criterion': ['gini', 'entropy'], 'max_depth': [None, 10, 20, 30], 'min_samples_split': [2, 5, 10]}
dt_grid_search = GridSearchCV(DecisionTreeClassifier(random_state=0), dt_param_grid, cv=5, scoring='accuracy')
dt_grid_search.fit(X_train_scaled, y_train)
dt_best_model = dt_grid_search.best_estimator_

# Print results for Decision Tree
print(f"Best Hyperparameters for Decision Tree: {dt_grid_search.best_params_}")

print_model_evaluation(dt_best_model, X_train_scaled, y_train, X_test_scaled, y_test, 'Decision Tree')
```

However, they may be prone to overfitting.

Naive Bayes Naive Bayes is a probabilistic algorithm based on Bayes' theorem with the assumption of independence between features. Naive Bayes is computationally efficient and works well with high-dimensional datasets.

```
# Naive Bayes (No hyperparameter tuning needed)
nb_model = GaussianNB()
print_model_evaluation(nb_model, X_train_scaled, y_train, X_test_scaled, y_test, 'Naive Bayes')

from sklearn.ensemble import VotingClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier
```

Bagging with Random Forest: Bagging (Bootstrap Aggregating) is an ensemble method that builds multiple models independently and combines their predictions. In this case, Random Forest is used as the base model. Bagging with Random Forest helps reduce overfitting and variance. It provides a more robust and accurate model by aggregating predictions from multiple decision trees.

```

# Define the base model (Random Forest)
base_model_rf = RandomForestClassifier(random_state=0)

# Bagging with Random Forest
bagging_rf_model = BaggingClassifier(base_estimator=base_model_rf, random_state=0)

# Hyperparameter tuning using GridSearchCV
param_grid_bagging_rf = {
    'n_estimators': [50, 100, 200],
    'max_samples': [0.5, 0.7, 1.0],
    'max_features': [0.5, 0.7, 1.0]
}

grid_search_bagging_rf = GridSearchCV(estimator=bagging_rf_model, param_grid=param_grid_bagging_rf, cv=5, scoring='accuracy', n_jobs=1)

grid_search_bagging_rf.fit(X_train_scaled, y_train)

# Get the best model from hyperparameter tuning
best_bagging_rf_model = grid_search_bagging_rf.best_estimator_

# Print results for the best Bagging model with Random Forest
print(f"Best Hyperparameters for Bagging with Random Forest: {grid_search_bagging_rf.best_params_}")

print_model_evaluation(best_bagging_rf_model, X_train_scaled, y_train, X_test_scaled, y_test, ' Bagging (Random Forest)')

```

Stacking: Stacking is an ensemble learning technique that combines predictions from multiple base models using a meta-model.

```

# Stacking with RandomForest as the final estimator
stacking_model = StackingClassifier(
    estimators=[
        ('RandomForest', rf_best_model),
        ('LogisticRegression', lr_best_model),
        ('KNN', knn_best_model),
        ('SVM', svm_best_model),
        ('DecisionTree', dt_best_model)
    ],
    final_estimator=RandomForestClassifier(random_state=0),
    stack_method='auto', # 'auto' to automatically select stacking method
    n_jobs=-1 # Use all processors
)

# Define hyperparameter grid for Stacking model
stacking_param_grid = {
    'final_estimator__n_estimators': [50, 100, 200],
    'final_estimator__max_depth': [None, 10, 20],
    'final_estimator__min_samples_split': [2, 5, 10]
}

# Create GridSearchCV for Stacking model
stacking_grid_search = GridSearchCV(stacking_model, stacking_param_grid, cv=5, scoring='accuracy', n_jobs=-1)
stacking_grid_search.fit(X_train_scaled, y_train)

# Get the best Stacking model from hyperparameter tuning
best_stacking_model = stacking_grid_search.best_estimator_

# Print results for the best Stacking model
print(f"Best Hyperparameters for Stacking: {stacking_grid_search.best_params_}")

print_model_evaluation(best_stacking_model, X_train_scaled, y_train, X_test_scaled, y_test, ' Stacking')

```

Stacking leverages the strengths of different models by combining them, potentially resulting in improved performance. It's useful when individual models capture different aspects of the data.

Boosting with Gradient Boosting: Boosting is an ensemble technique that combines weak learners to create a strong learner. Gradient Boosting is a specific boosting algorithm that builds trees sequentially, with each tree correcting the errors of the previous one. Boosting especially with Gradient Boosting, is powerful for improving predictive performance. It sequentially corrects errors and focuses on instances that were misclassified by earlier models.

```
# Gradient Boosting as the base model for Boosting
base_model_gb = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=0)

# Hyperparameter tuning using GridSearchCV
param_grid_gb = {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 5, 7]
}

grid_search_gb = GridSearchCV(base_model_gb, param_grid_gb, cv=5, scoring='accuracy', n_jobs=1)
grid_search_gb.fit(X_train_scaled, y_train)

# Get the best model from hyperparameter tuning
best_gradientboost_model = grid_search_gb.best_estimator_

# Print results for the best Gradient Boosting model
print(f"Best Hyperparameters for Gradient Boosting: {grid_search_gb.best_params}")

print_model_evaluation(best_gradientboost_model, X_train_scaled, y_train, X_test_scaled, y_test, ' Boosting (Gradient Boosting)')
```

XGBoost: (Extreme Gradient Boosting) is an optimized and scalable version of gradient boosting. It incorporates regularization terms to control model complexity. XGBoost is known for its speed and performance. Good efficiency, accuracy, and

ability to handle large datasets.

```
# Encode the target variable
label_encoder = LabelEncoder()
y_train_encoded = label_encoder.fit_transform(y_train)
y_test_encoded = label_encoder.transform(y_test)

# XGBoost model
xgboost_model = XGBClassifier(random_state=0)

# Define hyperparameter grid for XGBoost model
xgboost_param_grid = {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 5, 7],
    'min_child_weight': [1, 3, 5]
}

# Create GridSearchCV for XGBoost model
xgboost_grid_search = GridSearchCV(xgboost_model, xgboost_param_grid, cv=5, scoring='accuracy', n_jobs=-1)
xgboost_grid_search.fit(X_train_scaled, y_train_encoded)

# Get the best XGBoost model from hyperparameter tuning
best_xgboost_model = xgboost_grid_search.best_estimator_

# Print results for the best XGBoost model
print(f"Best Hyperparameters for XGBoost: {xgboost_grid_search.best_params_}")

print_model_evaluation(best_xgboost_model, X_train_scaled, y_train_encoded, X_test_scaled, y_test_encoded, ' XGBoost')
```

-All the above models were created together with to computed hyperparameter tuning and cross validation to optimize its performance -

Individual Model Training and Hyperparameter Tuning:

The hyperparameter tuning process involves finding the best set of hyperparameters for a machine learning model to optimize its performance. GridSearchCV (Grid Search Cross-Validation) is a technique that systematically tests a predefined set of hyperparameter values to identify the combination that yields the best model performance. For example,

Random Forest hyperparameter tuning with GridSearchCV:

Hyperparameters for Random Forest include **n_estimators** (number of trees in the forest), **max_depth** (maximum depth of each tree), and **min_samples_split** (minimum number of samples required to split an internal node).

Define the Hyperparameter Grid: Specify the hyperparameter values to be tested, for Random Forest, we might define a grid of values for **n_estimators**, **max_depth**, and **min_samples_split**. Create an instance of the Random Forest model and the

GridSearchCV object. Pass the model, hyperparameter grid, and evaluation metric (accuracy) to GridSearchCV. Fit the GridSearchCV object, to the training data. This will perform cross-validated grid search, training and evaluating the model for each combination of hyperparameters.

##there is no hyperparameter tuning in Naive Bayes. It is an exception due to its simplicity and the limited number of hyperparameters. ##

Types of **ensemble models** I tried including:

- **Bagging (Bootstrap Aggregating):** Training multiple instances of the same model on different subsets of the training data and averaging their predictions. Example(Random Forest).
- **Boosting:** Sequentially training models, where each subsequent model focuses on correcting the errors of the previous ones. Gradient Boosting, Adaboosting and XGBoost are common examples.
- **Stacking:** Training multiple diverse models and combining their predictions using another model (meta-model). Stacking can capture more complex patterns by learning how to weigh the predictions of the base models.

-Afterwards, Present the best hyperparameters and evaluate the model using accuracy, precision, recall, and F1 score. Logistic Regression, K-Nearest Neighbors, SVM, Decision Tree, Naive Bayes, Random Forest, Gradient Boosting, stacking and XGBoost. I repeated the hyperparameter tuning process for each individual model and presented the results.

Feature Importance Analysis: Feature Importance Analysis is to understand the contribution of different features (variables) in a machine learning model's decision-making process. It helps identify which features have a significant impact on the model's predictions.

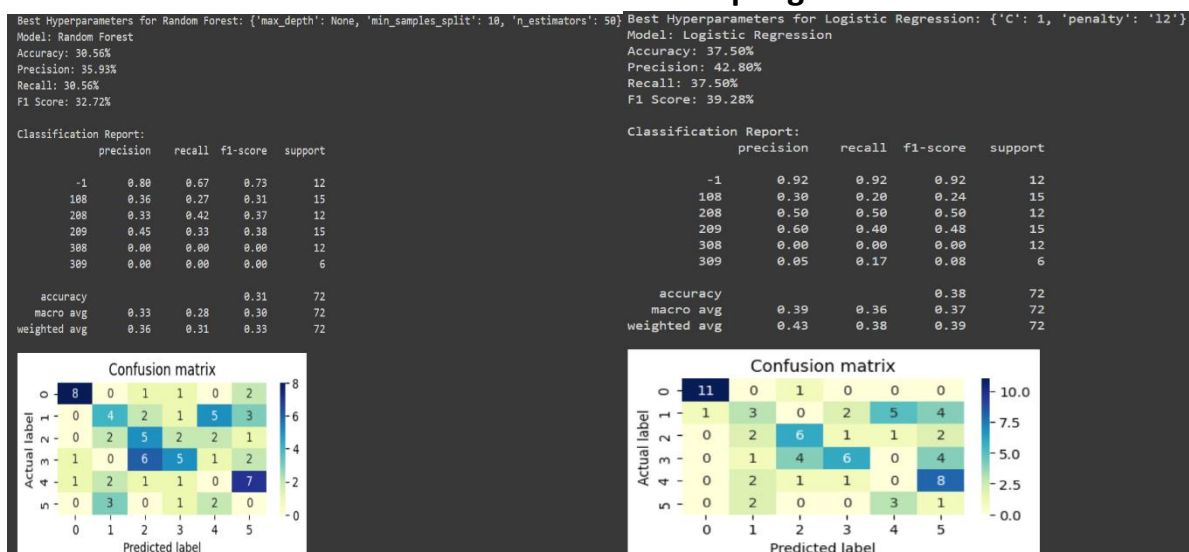
1. **Random Forest:** Feature importance in a Random Forest model are determined by how much each feature contributes to the reduction in impurity (Gini impurity) when making decisions. Feature importance are normalized and ranked, indicating the relative importance of each feature.

2. **Logistic Regression:** Coefficients in logistic regression represent the change in the log-odds of the response variable for a one-unit change in the predictor variable. Larger absolute coefficients suggest a more influential feature.
3. **Bagging with Random Forest:** Similar to the individual Random Forest model, feature importances are derived from impurity reduction. Features with high importances across multiple base models are considered more robust.
4. **Stacking:** Feature importances can be obtained from the final estimator (e.g., another Random Forest). Analyzing importances at both base model and meta-model levels provides insights into the overall stacking model.
5. **Boosting with Gradient Boosting and XGBoost:** Feature importances are derived from how often a feature is used for splitting and the improvement it brings to the model's performance. Features that contribute to reducing the loss function are considered more important.

Results and Discussion

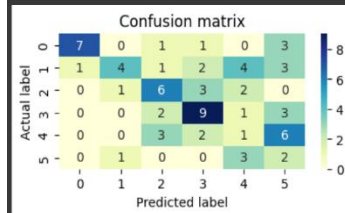
Summary of Findings: Tested in 3 imbalanced data handling ways, and got different accuracies. Below are the Highlights of best-performing models and their respective features.

Below are accuracies of models: -Undersampling-



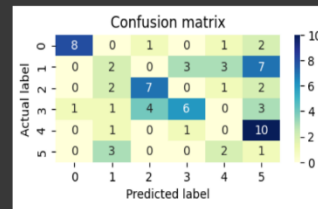
Best Hyperparameters for K-Nearest Neighbors: {'n_neighbors': 5, 'weights': 'distance'}
Model: K-Nearest Neighbors
Accuracy: 40.28%
Precision: 49.69%
Recall: 40.28%
F1 Score: 42.22%

	precision	recall	f1-score	support
-1	0.88	0.58	0.70	12
108	0.67	0.27	0.38	15
208	0.46	0.50	0.48	12
209	0.53	0.60	0.56	15
308	0.09	0.08	0.09	12
309	0.12	0.33	0.17	6
accuracy			0.40	72
macro avg	0.46	0.39	0.40	72
weighted avg	0.50	0.40	0.42	72



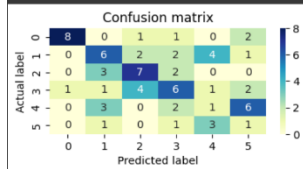
Best Hyperparameters for Support Vector Machine: {'C': 0.1, 'gamma': 'scale', 'kernel': 'linear'}
Model: Support Vector Machine
Accuracy: 33.33%
Precision: 42.00%
Recall: 33.33%
F1 Score: 36.43%

	precision	recall	f1-score	support
-1	0.89	0.67	0.76	12
108	0.22	0.13	0.17	15
208	0.58	0.58	0.58	12
209	0.60	0.40	0.48	15
308	0.00	0.00	0.00	12
309	0.04	0.17	0.06	6
accuracy			0.33	72
macro avg	0.39	0.33	0.34	72
weighted avg	0.42	0.33	0.36	72



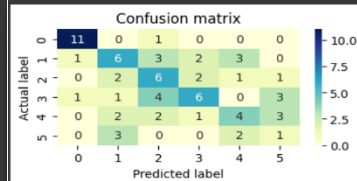
Best Hyperparameters for XGBoost: {'learning_rate': 0.1, 'max_depth': 3, 'min_child_weight': 1, 'subsample': 0.8}
Model: XGBoost
Accuracy: 40.28%
Precision: 43.55%
Recall: 40.28%
F1 Score: 41.43%

	precision	recall	f1-score	support
0	0.89	0.67	0.76	12
1	0.43	0.40	0.41	15
2	0.50	0.58	0.54	12
3	0.43	0.40	0.41	15
4	0.11	0.08	0.10	12
5	0.08	0.17	0.11	6
accuracy			0.40	72
macro avg	0.41	0.38	0.39	72
weighted avg	0.44	0.40	0.41	72



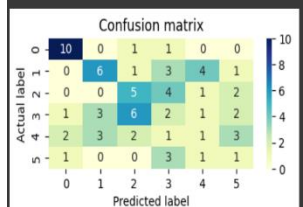
Best Hyperparameters for Stacking: {'final_estimator_max_depth': None, 'final_estimator': 'xgboost', 'n_estimators': 100, 'verbose': 0}
Model: Stacking
Accuracy: 47.22%
Precision: 48.35%
Recall: 47.22%
F1 Score: 47.30%

	precision	recall	f1-score	support
-1	0.85	0.92	0.88	12
108	0.43	0.40	0.41	15
208	0.38	0.50	0.43	12
209	0.55	0.40	0.46	15
308	0.40	0.33	0.36	12
309	0.12	0.17	0.14	6
accuracy			0.47	72
macro avg	0.45	0.45	0.45	72
weighted avg	0.48	0.47	0.47	72



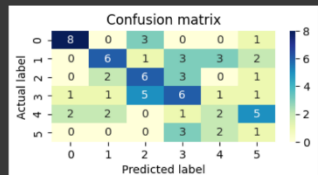
Best Hyperparameters for Decision Tree: {'criterion': 'gini', 'max_depth': None, 'min_samples_split': 5}
Model: Decision Tree
Accuracy: 34.72%
Precision: 33.86%
Recall: 34.72%
F1 Score: 33.90%

	precision	recall	f1-score	support
-1	0.71	0.83	0.77	12
108	0.50	0.40	0.44	15
208	0.33	0.42	0.37	12
209	0.14	0.13	0.14	15
308	0.12	0.08	0.10	12
309	0.11	0.17	0.13	6
accuracy			0.35	72
macro avg	0.32	0.34	0.33	72
weighted avg	0.34	0.35	0.34	72



Best Hyperparameters for Gradient Boosting: {'learning_rate': 0.2, 'max_depth': 5, 'n_estimators': 100, 'verbose': 0}
Model: Boosting (Gradient Boosting)
Accuracy: 40.28%
Precision: 42.89%
Recall: 40.28%
F1 Score: 41.00%

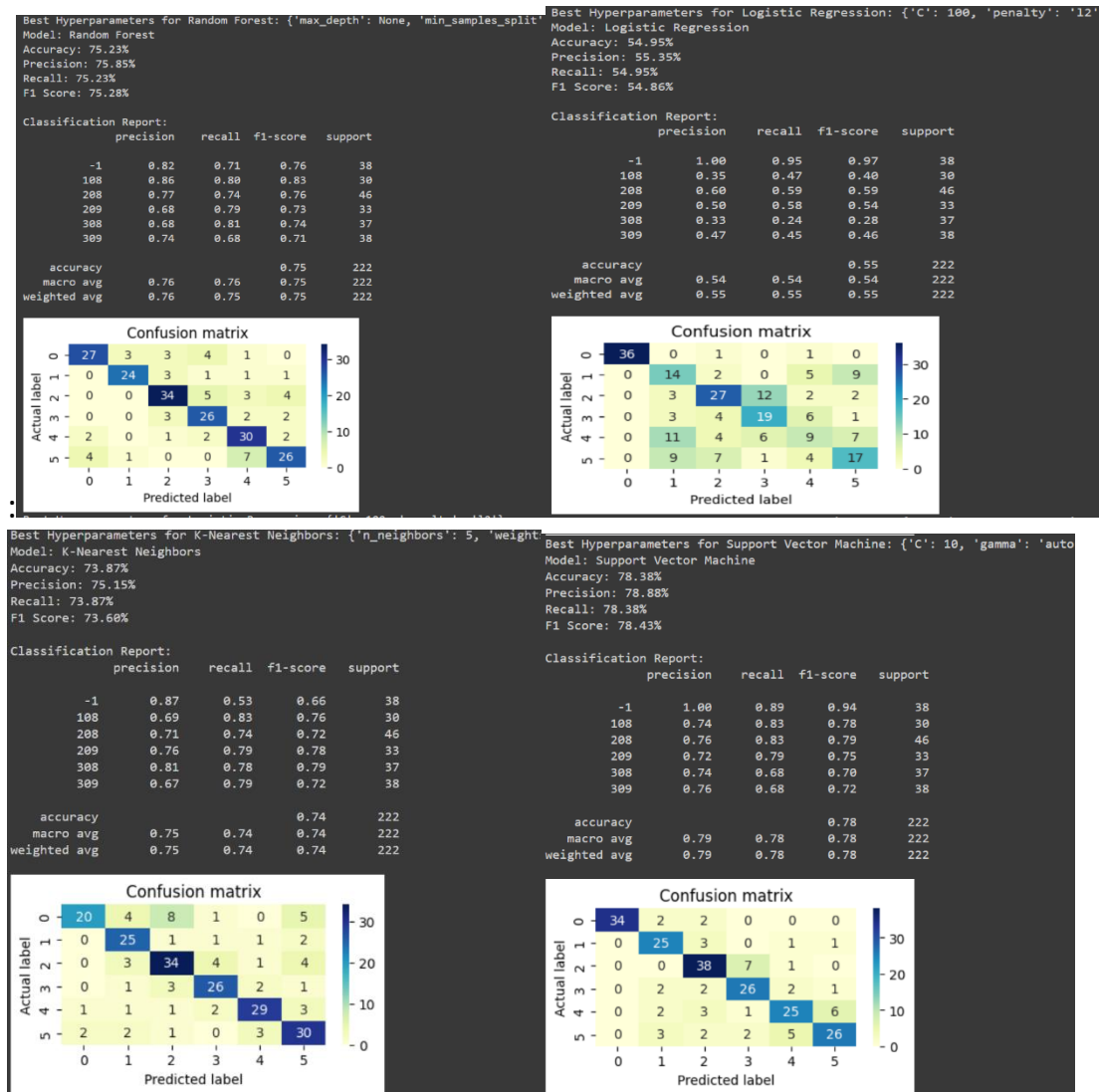
	precision	recall	f1-score	support
-1	0.73	0.67	0.70	12
108	0.55	0.40	0.46	15
208	0.40	0.50	0.44	12
209	0.38	0.40	0.39	15
308	0.25	0.17	0.20	12
309	0.09	0.17	0.12	6
accuracy			0.40	72
macro avg	0.40	0.38	0.38	72
weighted avg	0.43	0.40	0.41	72

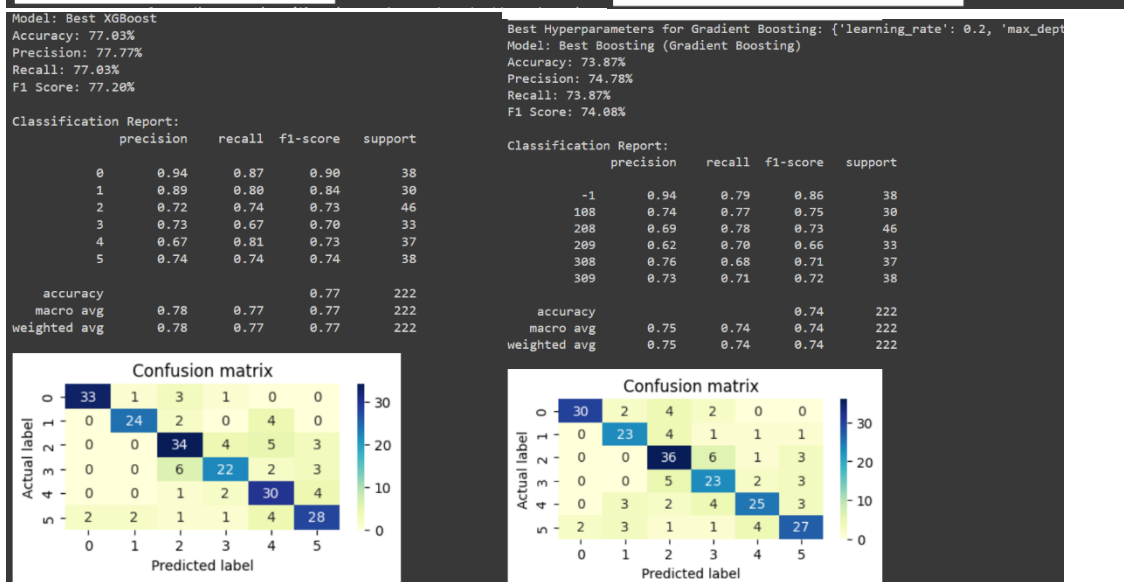
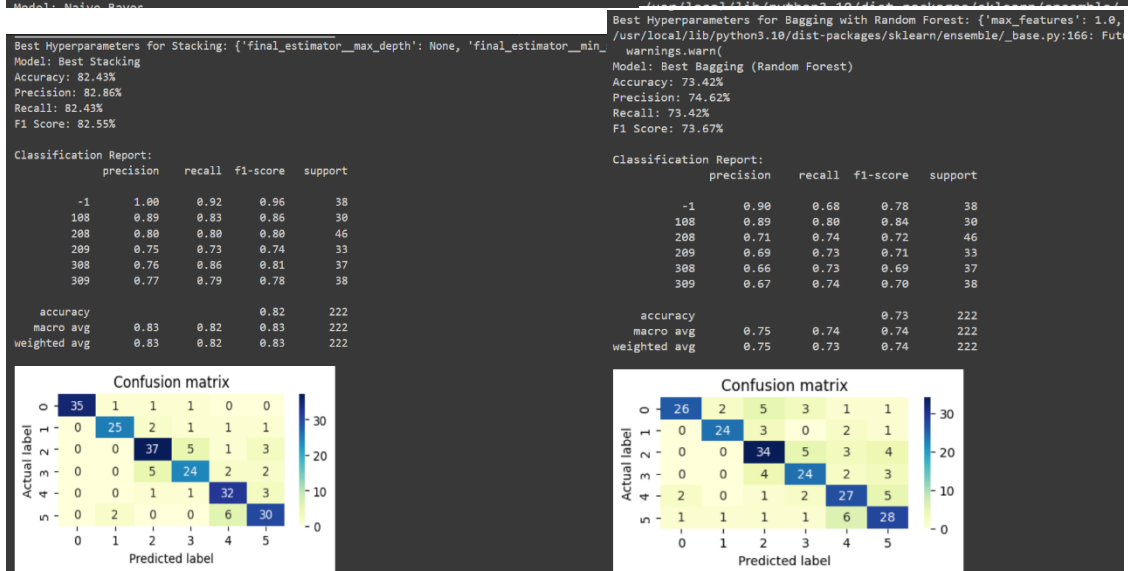
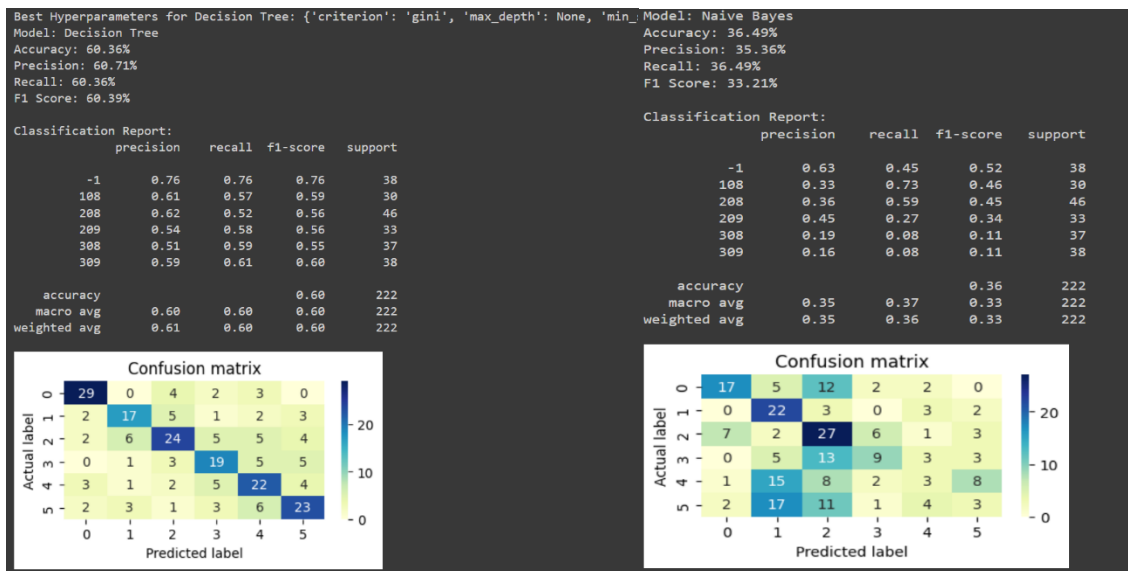


47.22% is the highest accuracy accomplished in under sampling method. and it is from an ensemble model stacking.

Below are the results from **-SMOTE-**

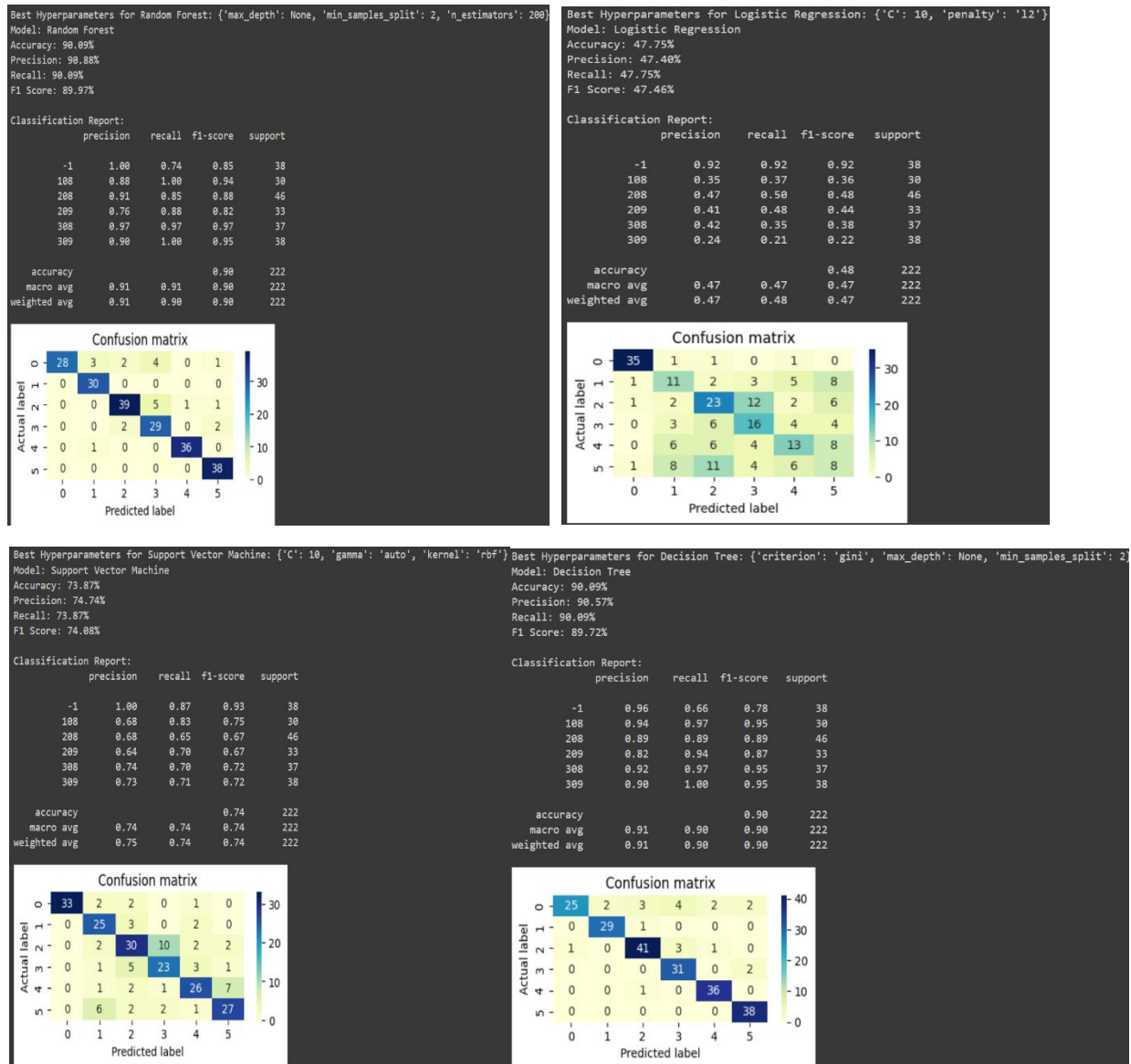
The highest accuracy is 82.43% from ensemble model stacking. Then 78.38% from support vector machine.





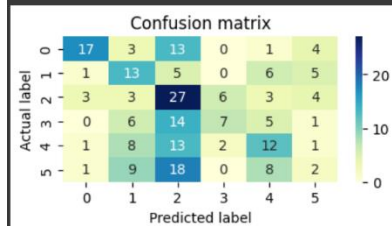
Accuracies from -Oversampling-

Highest accuracy is 94.14% from XGBoost and stacking. Then 90.09% from Random Forest classifier.



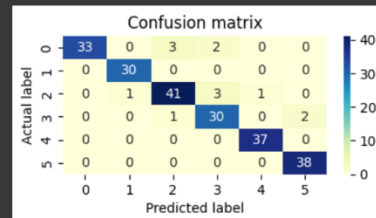
```
Model: Naive Bayes
Accuracy: 35.14%
Precision: 37.72%
Recall: 35.14%
F1 Score: 33.78%
```

Classification Report:				
	precision	recall	f1-score	support
-1	0.74	0.45	0.56	38
108	0.31	0.43	0.36	30
208	0.30	0.59	0.40	46
209	0.47	0.21	0.29	33
308	0.34	0.32	0.33	37
309	0.12	0.05	0.07	38
accuracy				222
macro avg	0.38	0.34	0.34	222
weighted avg	0.38	0.35	0.34	222



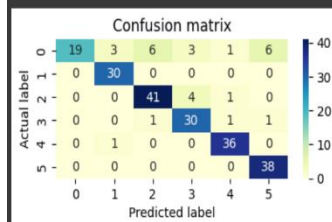
```
Best Hyperparameters for XGBoost: {'learning_rate': 0.1, 'max_depth': 7, 'min_child_weight': 1}
Model: XGBoost
Accuracy: 94.14%
Precision: 94.38%
Recall: 94.14%
F1 Score: 94.11%
```

Classification Report:				
	precision	recall	f1-score	support
0	1.00	0.87	0.93	38
1	0.97	1.00	0.98	30
2	0.91	0.89	0.90	46
3	0.86	0.91	0.88	33
4	0.97	1.00	0.99	37
5	0.95	1.00	0.97	38
accuracy			0.94	222
macro avg	0.94	0.94	0.94	222
weighted avg	0.94	0.94	0.94	222



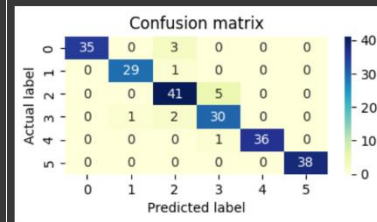
```
Best Hyperparameters for K-Nearest Neighbors: {'n_neighbors': 10, 'weights': 'distance'}
Model: K-Nearest Neighbors
Accuracy: 87.39%
Precision: 88.63%
Recall: 87.39%
F1 Score: 86.36%
```

Classification Report:		precision	recall	f1-score	support
-1	1.00	0.59	0.67	38	
108	0.88	1.00	0.94	30	
208	0.85	0.89	0.87	46	
209	0.81	0.91	0.86	33	
308	0.92	0.97	0.95	37	
309	0.84	1.00	0.92	38	
accuracy				0.87	222
macro avg		0.89	0.88	0.87	222
weighted avg		0.89	0.87	0.86	222



```
Best Hyperparameters for Stacking: {'final_estimator__max_depth': 10, 'final_estimator  
Model: Stacking  
Accuracy: 94.14%  
Precision: 94.43%  
Recall: 94.14%  
F1 Score: 94.23%
```

Classification	Report: precision	recall	f1-score	support
-1	1.00	0.92	0.96	38
108	0.97	0.97	0.97	30
208	0.87	0.89	0.88	46
209	0.83	0.91	0.87	33
308	1.00	0.97	0.99	37
309	1.00	1.00	1.00	38
accuracy			0.94	222
macro avg	0.95	0.94	0.94	222
weighted avg	0.94	0.94	0.94	222



Conclusion

By looking in to the results the Highest accuracy is given by XGBoost and stacking which is 94.14%. Then 90.09% from Random Forest classifier. Also KNN ,SVM have given good accuracies.

Lastly from the best-performing models I extracted their respective features.

```
# Feature importances from XGBoost
feature_importances_xgboost = best_xgboost_model.feature_importances_

# Create a DataFrame to display feature importances
feature_importance_df_xgboost = pd.DataFrame({'Feature': X.columns, 'Importance': feature_importances_xgboost})

# Sort the DataFrame by importance in descending order
feature_importance_df_xgboost = feature_importance_df_xgboost.sort_values(by='Importance', ascending=False)

# Display the sorted feature importances
print("Feature Importances - XGBoost:")
print(feature_importance_df_xgboost)

# Calculate permutation importances
perm_importance = permutation_importance(svm_best_model, X_test_scaled, y_test, n_repeats=30, random_state=0)

# Create a DataFrame to display feature importances
feature_importance_df_permutation = pd.DataFrame({'Feature': X.columns, 'Importance': perm_importance.importances_mean})

# Sort the DataFrame by importance
feature_importance_df_permutation = feature_importance_df_permutation.sort_values(by='Importance', ascending=False)

# Display the sorted feature importances
print("Permutation Feature Importances - Support Vector Machine (Non-Linear):")
print(feature_importance_df_permutation[['Feature', 'Importance']])

# Feature importances from Random Forest
feature_importances = rf_best_model.feature_importances_

# Create a DataFrame to display feature importances
feature_importance_df = pd.DataFrame({'Feature': X.columns, 'Importance': feature_importances})

# Sort the DataFrame by importance in descending order
feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)

# Display the sorted feature importances
print("Feature Importances:")
print(feature_importance_df)
```


Feature Importances - XGBoost:			Permutation Feature Importances - Support Vector Machine (Non-Linear):		
	Feature	Importance		Feature	Importance
13	HRV_ApEn	0.143374	13	HRV_ApEn	0.293544
14	HRV_SampEn	0.089804	14	HRV_SampEn	0.216216
6	HRV_HTI	0.071768	11	HRV_DFA_alpha1	0.155105
3	HRV_Prc20NN	0.068116	12	HRV_DFA_alpha2	0.153003
2	HRV_RMSSD	0.064705	1	HRV_SDNN	0.137087
12	HRV_DFA_alpha2	0.062602	6	HRV_HTI	0.099850
1	HRV_SDNN	0.061395	3	HRV_Prc20NN	0.096096
0	HRV_MeanNN	0.061199	9	HRV_LFHF	0.083634
8	HRV_TP	0.058431	4	HRV_Prc80NN	0.081832
5	HRV_pNN50	0.057829	10	HRV_SD1SD2	0.067718
11	HRV_DFA_alpha1	0.057740	0	HRV_MeanNN	0.066817
10	HRV_SD1SD2	0.057171	5	HRV_pNN50	0.062162
7	HRV_VLF	0.051159	7	HRV_VLF	0.042342
9	HRV_LFHF	0.049625	8	HRV_TP	0.030931
4	HRV_Prc80NN	0.045082	2	HRV_RMSSD	0.027628

Feature Importances:		
	Feature	Importance
13	HRV_ApEn	0.154620
14	HRV_SampEn	0.084693
12	HRV_DFA_alpha2	0.074319
10	HRV_SD1SD2	0.066475
6	HRV_HTI	0.064800
11	HRV_DFA_alpha1	0.061652
1	HRV_SDNN	0.060216
3	HRV_Prc20NN	0.058856
7	HRV_VLF	0.057943
9	HRV_LFHF	0.057278
8	HRV_TP	0.054844
2	HRV_RMSSD	0.052495
0	HRV_MeanNN	0.051957
4	HRV_Prc80NN	0.051224
5	HRV_pNN50	0.048629

<-Feature importance of random forest

So, while considering the 3 class imbalance handling strategies the random over sampling showed the best accuracies in the models. Among the models which gave high accuracies, the highest accuracy is given by XGBoost and stacking models which is 94.14%. also we can consider that the features HRV_ApEn (à Approximate entropy, which is a technique used to quantify the amount of regularity and the unpredictability of fluctuations over time-series data) HRV_HT1 And HRV_SampEn(à Sample Entropy) are the best features to build the best model able to predict the different emotional states.

